

Assignment 1

1. The following is the entity statement for a decoder.

```
entity decoder is
generic ( N: integer :=2; M integer :=4;)
port ( enable: in STD_LOGIC;
      address: in STD_LOGIC_VECTOR (N-1 downto 0);
      pointers: out STD_LOGIC_VECTOR (M-1 downto 0));
end decoder;
```

Draw a block diagram for the decoder. In your diagram use arrows appropriately to show inputs and outputs.

2. Following is a VHDL code that describes a 1-bit adder.

```
library IEEE;
use IEEE.std_logic_1164.all;
entity adder1bit is port (
    a,b,cp: in std_logic;
    s,c: out std_logic );
end adder1bit;
architecture ad of adder1bit is
    signal ab,bb,cb std_logic;
begin
    ab <= not a;
    bb <= not b;
    cb <= not cp;
    s <= (ab and bb and cp) or (ab and b and cb) or
        (a and bb and cb) or (a and b and cp);
    c <= (ab and b and cp) or (a and bb and cp) or (a and b);
end ad;
```

- (a) Draw a block diagram for the adder.
- (b) Write logic expressions for the two outputs *s* and *c* in terms of the inputs *a*, *b* and *cp*.
- (c) Following is a VHDL code for a two-bit adder which uses the above 1-bit adder as a component.

```
library IEEE;
use IEEE.std_logic_1164.all;
entity adder is port (
    a0, a1, b0, b1: in std_logic;
    s0,s1: out std_logic );
end adder;
architecture add of adder is
    component adder1bit
        port(a,b,cp: in s,p: out);
    end component;
    signal c0,c1,cpr std_logic ;
begin
    cpr <= '0';
    x1:adder1bit port map(a0,b0,cpr,s0,c0);
    x2:adder1bit port map(a1,b1,c0,s1,c1);
end add;
```

Draw a diagram to show how the 1-bit adders are connected together to form the 2-bit adder.

3. You are given two VHDL files named mux.vhdl and mux_tb.vhdl. Perform a simulation of the circuit mux.vhdl using Vivado. Submit the resulting timing diagram.
4. Draw a timing diagram to show the test bench signals generated by the following VHDL code. (Take CLK_I_period as 10 ns.)

```

-- Clock process definitions
CLK_I_process :process
begin
    CLK_I <= '0';
    wait for CLK_I_period/2;
    CLK_I <= '1';
    wait for CLK_I_period/2;
end process;
-- Stimulus process
stim_proc: process
begin
    RST_I <= '1';
    wait for 17 ns;
    RST_I <= '0';
    wait for 20 ns;
    RST_I <= '1';
    wait for 10 ns;
    RST_I <= '0';
    wait;
end process;

```